SOI 2025 Fraud Detection Challenge

Team: The Pheonix
Author: Vinit
Date: July 2025

🌀Introduction

In the SOI 2025 Fraud Detection Challenge, we were tasked with building a high-performance binary classification system to detect fraudulent behavior. The dataset was real-world in nature, consisting of 1,000,000 records with 32 features, some of which contained noise, missing values, and class imbalance. Only ~1.1% of the entries were fraud cases, making this an extreme imbalance problem.

🏜️Data Exploration & Cleaning

- **Missing and Invalid Values:**
- Replaced '-1' with NaN in columns known to represent missing data.
- Imputed all NaNs with the **median** of each column.
- **Constant Columns:**
- Removed constant column: `device_fraud_count`
- **Outlier Handling:**
- Capped all numeric columns at the **99.5th percentile** to reduce the influence of extreme values.
- **Categorical Encoding:**
- One-hot encoded the following features: `payment_type`, `employment_status`, `housing_status`, `source`, and `device_os`
- **Feature Engineering:**
- Created new features: `feature_sum`, `feature_mean`, `feature_std`, `feature_max`, `feature_min`, `feature_range` from existing numeric columns.

☃️Modeling Strategy

We chose to train **two fundamentally different models**:

1. Logistic Regression

- Linear model that is interpretable and sensitive to **global trends**.

- Good at capturing **additive effects** and performs well when paired with feature scaling.

2. XGBoost (GPU-accelerated)

- Powerful non-linear model that learns **hierarchical splits**.
- Excellent for capturing complex **interactions** between variables.

⚠️ Instead of using two tree-based models, we deliberately combined a **linear model** and a **tree-based model** to capture different types of information from the data.

🪻 Hyperparameter Tuning (Optuna)

Both models were tuned using **Optuna** to search for the best hyperparameters that maximize **F1-score**, which is appropriate for the imbalanced nature of the dataset.

Logistic Regression

- **Best Parameters:**
  `{'C': 1.7007, 'class_weight': 'balanced'}`
- **Best Threshold:** 0.9300
- **Best F1-score:** 0.2092

XGBoost

- **Best Parameters:**

```
{
  "max_depth": 3,
  "learning_rate": 0.0231,
  "subsample": 0.8015,
  "colsample_bytree": 0.5691,
  "reg_alpha": 7.8275,
  "reg_lambda": 4.6351,
  "scale_pos_weight": 10.7609
}
```

- **Best Threshold:** 0.5100
- **Best F1-score:** 0.2430

🐃Evaluation Results

Logistic Regression

```
              precision    recall  f1-score   support

           0     0.9913    0.9905    0.9909    197794
           1     0.2022    0.2167    0.2092      2206
```

```
              accuracy                           0.9819    200000
             macro avg     0.5967    0.6036    0.6000    200000
          weighted avg     0.9826    0.9819    0.9822    200000
```

XGBoost

```
              precision    recall  f1-score   support

           0     0.9922    0.9870    0.9895    197794
           1     0.2042    0.3001    0.2430      2206

    accuracy                         0.9794    200000
   macro avg     0.5982    0.6435    0.6163    200000
weighted avg     0.9835    0.9794    0.9813    200000
```

🦛Saved Models

- `logistic_regression_model.pkl`
- `scaler.pkl`
- `xgboost_model.json`

These models can be directly used for production-level inference and submission pipelines.

🐃Future Work

- Try **ensemble methods** like stacking or blending to combine LR and XGBoost
- Explore **feature selection** via SHAP values to reduce dimensionality
- Try **semi-supervised learning** or anomaly detection techniques
- Use **Isotonic or Platt calibration** to improve probability calibration
- Experiment with **SMOTE or ADASYN** for oversampling the minority class

---

🌟Conclusion

This project demonstrates a robust end-to-end fraud detection pipeline: - Cleaned, transformed, and feature-engineered a noisy real-world dataset - Tuned Logistic Regression and XGBoost for F1 optimization - Combined interpretability and performance by choosing diverse model types - Delivered reproducible results with model saving and threshold tuning

> Team *The Pheonix* (Vinit) has shown that smart modeling choices and precision tuning beat brute force — always.